**CENTRO UNIVERSITÁRIO UNIÃO DAS AMÉRICAS – UNIAMÉRICA**
**DESCOMPLICA**
**CURSO DE GRADUAÇÃO EM ENGENHARIA DE SOFTWARE**

Lilian Ramires Lima Castanha
Marcelo Nicolás Luján Villalba

# CLASSIFICAÇÃO DE TEXTOS COM A UTILIZAÇÃO DE GRANDES MODELOS DE LINGUAGEM: ESTUDO EXPERIMENTAL COM RELATOS DE ERROS REGISTRADOS EM SISTEMAS DE HELP DESK DE APOIO À IMPLANTAÇÃO/PRODUÇÃO DE SOFTWARES EDUCACIONAIS

Foz do Iguaçu/PR
2026

Lilian Ramires Lima Castanha
Marcelo Nicolás Luján Villalba

# CLASSIFICAÇÃO DE TEXTOS COM A UTILIZAÇÃO DE GRANDES MODELOS DE LINGUAGEM: ESTUDO EXPERIMENTAL COM RELATOS DE ERROS REGISTRADOS EM SISTEMAS DE HELP DESK DE APOIO À IMPLANTAÇÃO/PRODUÇÃO DE SOFTWARES EDUCACIONAIS

Projeto de Pesquisa apresentado à componente curricular de Projeto Final de Curso I do Bacharelado em Engenharia de Software do Centro Universitário União das Américas – Uniamérica Descomplica como requisito parcial para obtenção do grau de Bacharel em Engenharia de Software.

Orientadora: Prof.ª DSc. Isabel Fernandes de Souza

Foz do Iguaçu/PR
2026

Lilian Ramires Lima Castanha
Marcelo Nicolás Luján Villalba

**CLASSIFICAÇÃO DE TEXTOS COM A UTILIZAÇÃO DE GRANDES MODELOS DE LINGUAGEM: ESTUDO EXPERIMENTAL COM RELATOS DE ERROS REGISTRADOS EM SISTEMAS DE HELP DESK DE APOIO À IMPLANTAÇÃO/PRODUÇÃO DE SOFTWARES EDUCACIONAIS**

Aprovado em: __/__/__

Banca Examinadora

---

Prof.Dr. Xxxx Xxxx Xxxx Xxxx
Presidente da banca

---

Nome do Professor (com titulação)
Examinador
Instituição

---

Nome do Professor (com titulação)
Examinador
Instituição

## RESUMO

Este trabalho apresenta um estudo experimental sobre a utilização de Grandes Modelos de Linguagem (Large Language Models – LLMs) na classificação automática de relatos de erros registrados em sistemas de Help Desk voltados a softwares educacionais. A pesquisa propõe o desenvolvimento da API LinkNix, uma plataforma capaz de integrar modelos de inteligência artificial generativa, como GPT, Claude e DeepSeek, utilizando técnicas de engenharia de prompt parametrizável para automatizar o processo de triagem e categorização de chamados. A metodologia adotada possui abordagem aplicada, exploratória e quanti-qualitativa, sendo estruturada por meio de revisão integrativa da literatura, modelagem arquitetural da solução e desenvolvimento experimental do protótipo. A plataforma foi projetada utilizando arquitetura baseada em API REST, FastAPI, PostgreSQL e mecanismos de observabilidade para monitoramento de métricas relacionadas ao consumo de tokens, custos operacionais, desempenho e assertividade das classificações realizadas pelos modelos de IA. Espera-se que os resultados contribuam para a otimização do processo de manutenção de software, redução do esforço manual de triagem de bugs e ampliação das pesquisas sobre a aplicação de LLMs na Engenharia de Software educacional.



**Palavras-chave:** INTELIGÊNCIA ARTIFICIAL. ENGENHARIA DE SOFTWARE. BUG TRACKING. HELP DESK. LARGE LANGUAGE MODELS.

**SUMÁRIO**


1 INTRODUÇÃO ..... 7
2 TEMA ..... 9
3 PROBLEMA ..... 10
4 HIPÓTESES ..... 11
5 OBJETIVOS ..... 12
5.1 OBJETIVO GERAL ..... 12
5.2 OBJETIVOS ESPECÍFICOS ..... 12
6 JUSTIFICATIVA ..... 13
7 REFERENCIAL TEÓRICO ..... 14
7.1 MANUTENÇÃO DE SOFTWARE: CONCEITOS E DESAFIOS ..... 14
7.2 RASTREAMENTO DE BUGS: DEFINIÇÕES E ABORDAGENS ..... 14
7.3 GRANDES MODELOS DE LINGUAGEM (LLMS): FUNDAMENTOS ..... 16
7.4 LLMS APLICADOS À ENGENHARIA DE SOFTWARE ..... 16
7.5 SOFTWARE EDUCACIONAL: CARACTERÍSTICAS E ESPECIFICIDADES 17
7.6 EVIDÊNCIAS TEÓRICAS: LLMS NO RASTREAMENTO DE BUGS EM SOFTWARE EDUCACIONAL ..... 18
8 METODOLOGIA ..... 20
8.1 CLASSIFICAÇÃO DA PESQUISA ..... 20
8.2 PROCEDIMENTOS METODOLÓGICOS ..... 21
8.3 POPULAÇÃO E AMOSTRA ..... 22
8.4 INSTRUMENTOS DE COLETA DE DADOS ..... 23
8.5 PROCEDIMENTOS DE ANÁLISE DOS DADOS ..... 23
8.6 ENGENHARIA DE PROMPT ..... 24
8.7 PROCEDIMENTOS DE ANÁLISE DOS RESULTADOS ..... 24
8.8 JUSTIFICATIVA DA METODOLOGIA ..... 25
8.9 ARQUITETURA DA SOLUÇÃO ..... 25
8.9.1 Arquitetura geral da plataforma ..... 26
8.9.2 Sistema clientes LinkNix ..... 26
8.9.3 Sistemas clientes (Help Desk) ..... 26
8.9.4 API Gateway ..... 27
8.9.5 Prompt Engine ..... 27
8.9.6 Integração com grandes modelos de linguagem (LLMs) ..... 27
8.9.7 Banco de dados ..... 28
8.9.8 Módulo de métricas e observabilidade ..... 28
8.9.9 Dashboard analítico ..... 28

8.9.10 Fluxo operacional da classificação ..... 29
8.9.11 Considerações arquiteturais ..... 29
8.10 MODELAGEM UML ..... 29
8.10.1 Diagrama de casos de uso ..... 30
8.10.2 Diagrama de classes ..... 31
8.11 REQUISITOS FUNCIONAIS E NÃO FUNCIONAIS ..... 32
9 CRONOGRAMA ..... 34
10 CUSTOS ..... 35
11 RESULTADOS ESPERADOS ..... 36
REFERÊNCIAS ..... 37

# 1 INTRODUÇÃO

A manutenção de software integra uma parte essencial do ciclo de vida dos sistemas, consumindo parcela significativa do esforço técnico e financeiro após a implementação inicial (PRESSMAN; MAXIM, 2016; SOMMERVILLE, 2019). De acordo com Pushparani et al. (2025), a manutenção corretiva — que envolve a identificação, rastreamento e correção de bugs — representa uma parcela significativa desse esforço, sendo intensificada pela crescente complexidade dos sistemas modernos e pela necessidade de respostas rápidas às demandas dos usuários. Santos et al. (2025) complementam que o rastreamento eficiente de bugs se tornou uma competência estratégica para organizações de software, especialmente considerando que defeitos não identificados ou mal categorizados podem resultar em custos exponenciais de manutenção e insatisfação dos usuários.

Em sistemas voltados para a educação — como ambientes virtuais de aprendizagem, plataformas de gestão acadêmica e soluções de ensino a distância — esse cuidado torna-se ainda mais crítico, pois qualquer falha técnica nessas plataformas pode impactar diretamente os processos pedagógicos, prejudicar a experiência do usuário e impossibilitar a continuidade das atividades institucionais (HOU et al., 2024).

Durante a implantação e utilização de softwares, incluindo os educacionais de interesse desta pesquisa, é comum o surgimento de relatos de erros originários de diferentes perfis de usuários, como estudantes, docentes e gestores. Esses relatos, frequentemente descritos em linguagem natural e de forma não estruturada, precisam ser analisados, classificados e encaminhados adequadamente às equipes responsáveis. Esse processo, conhecido como classificação ou triagem de bugs, representa uma etapa essencial para a organização do fluxo de manutenção. Segundo Santos et al. (2025), a diversidade de métodos, dados e métricas utilizadas na classificação de bugs torna o processo de análise e comparação dos resultados um desafio na área de Engenharia de Software.

Nos últimos anos, o avanço da Inteligência Artificial, especialmente dos Grandes Modelos de Linguagem (LLMs), tem ampliado significativamente as possibilidades de automação de tarefas de análise textual. Estudos recentes indicam que modelos baseados em deep learning apresentam resultados promissores na

predição e classificação de defeitos de software, embora ainda enfrentem limitações relacionadas à representação semântica dos dados (GIRAY et al., 2023). Além disso, revisões recentes apontam o crescente uso de LLMs em atividades da Engenharia de Software, como análise de requisitos, geração de código e suporte à manutenção (HOU et al., 2024).

Nesse contexto, modelos como GPT-4, Gemini, LLaMA e CodeLlama demonstram capacidade de interpretar descrições de problemas e identificar padrões semânticos complexos, sugerindo potencial aplicação no apoio à classificação automatizada de bugs. A utilização adequada desses modelos depende, contudo, da construção de instruções eficazes, evidenciando a importância da literacia de prompts, uma vez que a qualidade das respostas depende diretamente da formulação das instruções (MOURA; CARVALHO, 2023).

Apesar do crescimento das pesquisas sobre IA aplicada à Engenharia de Software, ainda se observam poucos estudos com a aplicação de LLMs especificamente no processo de classificação de bugs em softwares educacionais. Tal lacuna evidencia a necessidade de aprofundamento teórico e experimental nessa área, especialmente considerando as particularidades dos ambientes educacionais e a diversidade dos perfis de usuários envolvidos.

Dessa forma, o presente trabalho propõe investigar como os Large Language Models (LLMs) têm sido utilizados na classificação de textos de relatos de erros em softwares educacionais, avaliando as abordagens relatadas na literatura, suas contribuições, limitações e potencial de aplicação prática. Além disso, busca-se propor diretrizes e desenvolver um protótipo preliminar que contribua para a otimização do processo de triagem de falhas.

## 2 TEMA

A classificação de erros de software em ambientes de implantação e/ou em produção continua sendo uma tarefa crítica. Portanto, o presente projeto de pesquisa tem como foco o desenvolvimento de um estudo com a seguinte temática:

> Manutenção de software orientada por IA: Classificação de textos de relatos de erros, registrados em sistemas de Help Desk de apoio à implantação/produção de softwares educacionais, com a utilização de grandes modelos de linguagem.

## 3 PROBLEMA

A manutenção de software representa uma das fases mais críticas do ciclo de vida de sistemas, sendo responsável por parcela significativa do esforço total de desenvolvimento. Estudos clássicos indicam que atividades de manutenção podem consumir entre 60% e 80% dos recursos ao longo da vida útil de um software (PRESSMAN; MAXIM, 2016; SOMMERVILLE, 2019).

No domínio educacional, a manutenção assume relevância ainda maior. Plataformas como ambientes virtuais de aprendizagem e sistemas de gestão acadêmica precisam operar com alta disponibilidade, confiabilidade e usabilidade, atendendo a públicos heterogêneos compostos por estudantes, professores e gestores. Estudos recentes sobre qualidade de sistemas educacionais digitais destacam que esses fatores são determinantes para a efetividade das plataformas e para a experiência do usuário (HOU et al., 2024). A ocorrência de falhas nesses sistemas pode impactar diretamente processos pedagógicos e administrativos, comprometendo a continuidade das atividades institucionais.

Durante a fase de implantação de softwares educacionais, o volume de relatos de erros tende a aumentar, exigindo processos eficientes de triagem e encaminhamento de bugs. Entretanto, quando realizados de forma manual, esses processos podem tornar-se demorados, suscetíveis a inconsistências e dependentes da experiência individual dos profissionais envolvidos (PRESSMAN; MAXIM, 2016).

Assim, a classificação de bugs de softwares educacionais ainda é um problema que pode se beneficiar de soluções baseadas em recursos tecnológicos contemporâneos, como as LLMs. Portanto, o estudo organizado no presente projeto de pesquisa procurará responder às seguintes perguntas norteadoras:

> É possível a classificação de erros, relatados em período de implantação/produção de softwares educacionais, com o apoio de LLMs? Quais as evidências teóricas da efetividade das LLMs na classificação de erros?

## 4 HIPÓTESES

Considerando o avanço das aplicações de LLMs na Engenharia de Software e sua capacidade de processar linguagem natural e analisar código, pressupõe-se que esses modelos possam oferecer ganhos relevantes no processo de rastreamento de bugs, especialmente em contextos que envolvem grande volume de relatos textuais de erro.

**H1 –** A utilização de LLMs é viável para a classificação de textos de relatos de erros registrados em sistemas de Help Desk de apoio à implantação/produção de softwares educacionais.

# 5 OBJETIVOS

## 5.1 OBJETIVO GERAL

*Classificar relatos textuais de erros, com a utilização de LLMs (ChatGPT, Claude e DeepSeek), registrados em sistemas de Help Desk de apoio à implantação/produção de softwares educacionais.*

## 5.2 OBJETIVOS ESPECÍFICOS

**OE1 –** Conceituar o processo de classificação de erros (*bug tracking/bug triaging*) no contexto da manutenção de softwares educacionais.

**OE2 –** Identificar, por meio de revisão integrativa da literatura, evidências da utilização de LLMs na classificação de erros relatados durante os períodos de implantação e produção em softwares educacionais.

**OE3 –** Mapear lacunas de pesquisa identificadas na literatura e propor uma API protótipo (LinkNix) para classificação e triagem de erros em softwares educacionais utilizando LLMs.

**OE4 –** Definir características relevantes para a classificação de erros e elaborar prompts a serem testados em, pelo menos, três modelos de LLMs integrados à API LinkNix.

**OE5 –** Desenvolver a API LinkNix, incorporando um modelo de prompt para a classificação de textos de erros.

**OE6 –** Integrar a API LinkNix ao sistema de apoio Help Desk JEDi Educa.

**OE7 –** Divulgar os resultados da pesquisa no formato de artigo científico na Jornada de Iniciação Científica e de Empreendedorismo da Descomplica Uniamérica.

## 6 JUSTIFICATIVA

A crescente digitalização do setor educacional tem ampliado a dependência de plataformas tecnológicas para a realização de atividades acadêmicas, administrativas e pedagógicas. Nesse cenário, a estabilidade e a qualidade desses sistemas tornam-se fatores estratégicos para instituições de ensino e organizações de tecnologia educacional. Considerando que a manutenção representa uma parcela significativa do esforço total no ciclo de vida do software (SOMMERVILLE, 2019), torna-se essencial investigar mecanismos que possam otimizar processos relacionados à identificação e ao tratamento de falhas.

O rastreamento de bugs, etapa fundamental da manutenção, envolve a análise de relatos textuais, classificação de erros e priorização de correções. Trata-se de uma atividade que exige interpretação semântica e raciocínio técnico, características que dialogam diretamente com as capacidades oferecidas pelos Large Language Models. Dessa forma, a aplicação desses modelos nesse contexto apresenta potencial inovador e merece investigação estruturada.

A relevância desta pesquisa fundamenta-se em três dimensões principais:

**Do ponto de vista científico,** observa-se que, embora existam estudos sobre LLMs na Engenharia de Software, a aplicação específica no domínio de softwares educacionais ainda é pouco explorada, configurando uma lacuna relevante na literatura.

**Do ponto de vista prático,** os resultados poderão fornecer subsídios para equipes de desenvolvimento de instituições de ensino, empresas de EdTech e projetos de software educacional, auxiliando na tomada de decisão quanto à adoção de ferramentas baseadas em Inteligência Artificial.

**Do ponto de vista metodológico,** o estudo contribui para o avanço de modelos de avaliação de LLMs aplicados à manutenção de software, área que se encontra em consolidação e demanda critérios mais sistemáticos de análise.

Assim, a pesquisa apresenta potencial de contribuir tanto para o aprofundamento teórico quanto para aplicações práticas no contexto da Engenharia de Software educacional.

# 7 REFERENCIAL TEÓRICO

O referencial teórico deste trabalho foi construído por meio de uma revisão teórica integrativa, método que permite reunir, analisar e sintetizar perspectivas diversas da literatura com o objetivo de consolidar o estado do conhecimento sobre um tema. O foco desta revisão recai sobre as evidências teóricas da efetividade do rastreamento de bugs com o apoio de LLMs em softwares educacionais, organizando o referencial em eixos temáticos interconectados.

## 7.1 MANUTENÇÃO DE SOFTWARE: CONCEITOS E DESAFIOS

A manutenção de software compõe uma das fases mais críticas e custosas do ciclo de vida de sistemas computacionais. Pressman e Maxim (2016) definem manutenção como o conjunto de atividades necessárias para manter um sistema de software operacional e adaptado às necessidades dos usuários após sua implantação. Sommerville (2019) complementa essa definição ao destacar que a manutenção representa tipicamente entre 60% e 80% dos custos totais de um projeto de software ao longo de sua vida útil.

Segundo Pressman e Maxim (2016), a manutenção de software pode ser classificada em quatro categorias principais: manutenção corretiva (correção de defeitos), manutenção adaptativa (adaptação a novos ambientes), manutenção perfectiva (melhoria de funcionalidades existentes) e manutenção preventiva (antecipação de problemas futuros). Entre essas categorias, a manutenção corretiva, que envolve o rastreamento e correção de bugs, representa parcela significativa do esforço total.

Sommerville (2019) enfatiza que os desafios da manutenção de software são amplificados pela complexidade crescente dos sistemas, pela escassez de documentação adequada e pela alta rotatividade de equipes de desenvolvimento. Esses fatores tornam o processo de identificação e correção de defeitos particularmente custoso e demorado, demandando novas abordagens e ferramentas de apoio.

## 7.2 RASTREAMENTO DE BUGS: DEFINIÇÕES E ABORDAGENS

*Bug tracking* ou *bug triage*, é o processo sistemático de identificação, classificação, priorização e acompanhamento de defeitos em sistemas de software

(PUSHPARANI et al., 2025). Segundo Santos et al. (2025), esse processo é fundamental para garantir a qualidade do software e a satisfação dos usuários, uma vez que permite uma resposta organizada e eficiente aos problemas relatados.

Pushparani et al. (2025) apresentam uma revisão histórica da evolução dos sistemas de rastreamento de bugs, destacando a transição de abordagens manuais e baseadas em planilhas para sistemas automatizados e integrados. Os autores argumentam que, com o crescimento da complexidade dos softwares e o aumento do volume de relatórios de bugs, tornou-se imprescindível o desenvolvimento de ferramentas que possam auxiliar na gestão eficiente desses defeitos.

O processo de rastreamento de bugs tipicamente envolve as seguintes etapas: (1) submissão do relatório de bug pelos usuários ou testadores; (2) triagem inicial para verificar a validade e evitar duplicações; (3) classificação do bug quanto ao tipo, severidade e prioridade; (4) atribuição do bug a um desenvolvedor responsável; e (5) acompanhamento até a resolução e verificação da correção (YILMAZ; TOROSLU; KÖKSAL, 2024).

Estudos recentes têm explorado o uso de técnicas baseadas em aprendizado profundo para estruturar e classificar relatórios de bugs de forma mais eficiente. Revisões sistemáticas indicam que modelos baseados em deep learning e representações semânticas apresentam melhor desempenho na identificação de padrões e similaridades entre relatos, contribuindo para processos de triagem mais precisos e escaláveis (HOU et al., 2024).

Structural Contrastive Learning Based Automatic Bug Triaging), que utiliza aprendizado contrastivo estrutural para realizar a triagem automática de bugs com alta precisão. Yilmaz, Toroslu e Köksal (2024) apresentam um estudo comparativo de diferentes paradigmas de aprendizado aplicados à detecção de prioridade de bugs, demonstrando que abordagens baseadas em deep learning superam métodos tradicionais em diversas métricas de desempenho.

Shao, Xiao e Yu (2026) exploram uma abordagem linguística para a identificação de bugs de concorrência, demonstrando que padrões textuais específicos nos relatórios podem indicar a presença desse tipo particular de defeito. Abordagens contemporâneas têm focado na otimização do processo de triagem e resolução de bugs por meio de técnicas avançadas de aprendizado de máquina, destacando ganhos em eficiência e redução do tempo de correção, especialmente quando integradas a modelos de linguagem e análise semântica.

## 7.3 GRANDES MODELOS DE LINGUAGEM (LLMS): FUNDAMENTOS

Os Grandes Modelos de Linguagem (LLMs) representam uma das inovações mais significativas no campo da inteligência artificial nas últimas décadas. Esses modelos são sistemas de processamento de linguagem natural treinados em vastas quantidades de dados textuais, capazes de compreender, gerar e manipular linguagem humana de forma sofisticada (HOU et al., 2024).

Moura e Carvalho (2023) destacam a importância da literacia de prompts para potencializar o uso efetivo da inteligência artificial na educação. Os autores argumentam que a capacidade de formular instruções claras e contextualizadas é fundamental para extrair o máximo potencial dos LLMs, permitindo que esses sistemas forneçam respostas mais precisas e relevantes às necessidades específicas dos usuários.

Simão et al. (2025) expandem essa discussão ao explorar o conceito de Inteligência Artificial Agêntica na Educação, onde LLMs não apenas respondem a consultas, mas atuam de forma proativa e autônoma para auxiliar em processos educacionais complexos. Essa abordagem agêntica tem implicações significativas para a aplicação de LLMs em contextos técnicos, como o rastreamento automatizado de bugs.

A arquitetura dos LLMs contemporâneos baseia-se principalmente em transformadores (transformers), uma estrutura neural que permite o processamento paralelo de sequências de texto e a captura de dependências de longo alcance. Esses modelos são pré-treinados em grandes corpus de dados por meio de tarefas como predição da próxima palavra, desenvolvendo uma compreensão profunda de padrões linguísticos, conhecimento factual e raciocínio lógico.

## 7.4 LLMS APLICADOS À ENGENHARIA DE SOFTWARE

A aplicação de LLMs à Engenharia de Software representa uma área de pesquisa em rápida expansão. Hou et al. (2024) conduziram revisões sistemáticas da literatura que mapeiam o estado da arte dessa intersecção, identificando diversas aplicações promissoras: geração automática de código, documentação de software, detecção de vulnerabilidades de segurança e, crucialmente para este trabalho, análise e rastreamento de bugs.

Giray (2023) fornece uma análise crítica sobre o uso de deep learning na predição de defeitos de software, contextualizando os avanços dos LLMs dentro da evolução mais ampla das técnicas de aprendizado de máquina aplicadas à qualidade de software. O autor demonstra que modelos baseados em aprendizado profundo superam consistentemente abordagens tradicionais em métricas de precisão e recall, especialmente quando treinados em grandes volumes de dados históricos de projetos.

Koyuncu (2026) apresenta um estudo empírico sobre categorização de relatórios de bugs com granularidade fina utilizando LLMs e engenharia de prompts. O estudo demonstra que, por meio da formulação cuidadosa de instruções contextualizadas, é possível obter classificações de bugs significativamente mais precisas e detalhadas do que aquelas produzidas por sistemas tradicionais. Esse trabalho estabelece uma base metodológica importante para a aplicação de LLMs em tarefas específicas de rastreamento de bugs.

A revisão de Hou et al. (2024) identifica várias vantagens dos LLMs para a Engenharia de Software: (1) capacidade de compreender contexto técnico complexo expresso em linguagem natural; (2) habilidade de generalizar conhecimento entre diferentes domínios e projetos; (3) flexibilidade para se adaptar a novas tarefas por meio de fine-tuning ou prompt engineering; e (4) potencial para automatizar tarefas que tradicionalmente exigem expertise humana.

No entanto, os mesmos autores destacam desafios importantes: necessidade de grandes volumes de dados de treinamento, custos computacionais elevados, questões de interpretabilidade das decisões do modelo e riscos de reprodução de vieses presentes nos dados de treinamento. Essas considerações são particularmente relevantes ao aplicar LLMs em contextos críticos como o rastreamento de bugs em software educacional.

## 7.5 SOFTWARE EDUCACIONAL: CARACTERÍSTICAS E ESPECIFICIDADES

No domínio educacional, a manutenção de software assume relevância ainda maior. Plataformas como ambientes virtuais de aprendizagem e sistemas de gestão acadêmica precisam operar com alta disponibilidade, confiabilidade e usabilidade, atendendo a públicos heterogêneos compostos por estudantes, professores e gestores. Estudos recentes sobre a qualidade de sistemas educacionais digitais destacam que esses fatores são determinantes para a efetividade das plataformas e para a experiência do usuário (HOU et al., 2024).

Os softwares educacionais apresentam características específicas que influenciam diretamente o processo de rastreamento de bugs:

**a) Diversidade de usuários:** estudantes, professores e administradores com diferentes níveis de letramento digital interagem com o sistema, gerando relatórios de bugs com graus variados de precisão técnica;

**b) Contexto pedagógico:** bugs podem estar relacionados não apenas a falhas técnicas, mas também a problemas de design instrucional ou inadequação de recursos educacionais;

**c) Sazonalidade de uso:** períodos letivos concentram maior volume de uso e, consequentemente, maior incidência de bugs reportados, exigindo processos eficientes de triagem e priorização;

**d) Impacto na continuidade educacional:** bugs críticos precisam ser corrigidos rapidamente para evitar interrupções no processo de ensino-aprendizagem, demandando mecanismos ágeis de identificação e resolução.

Essas especificidades tornam o rastreamento de bugs em software educacional particularmente desafiador, mas também oferecem oportunidades para a aplicação de tecnologias avançadas como LLMs, que podem processar linguagem natural de usuários não técnicos e compreender contextos pedagógicos complexos.

## 7.6 EVIDÊNCIAS TEÓRICAS: LLMS NO RASTREAMENTO DE BUGS EM SOFTWARE EDUCACIONAL

A convergência entre LLMs, rastreamento de bugs e software educacional representa um campo emergente com forte potencial teórico e prático. A literatura analisada nesta revisão fornece evidências teóricas que sustentam a viabilidade e efetividade dessa aplicação.

Observa-se que diferentes abordagens vêm sendo propostas para automatizar o rastreamento e a classificação de bugs, com resultados progressivamente mais robustos. Enquanto técnicas tradicionais baseadas em aprendizado de máquina apresentaram avanços iniciais, estudos mais recentes indicam que modelos baseados em aprendizado profundo e, especialmente, os Large Language Models, oferecem maior capacidade de interpretação semântica e generalização.

Os trabalhos analisados convergem ao evidenciar que a utilização de LLMs pode melhorar significativamente a precisão na classificação de relatos de bugs, sobretudo em cenários que envolvem linguagem natural não estruturada. Além disso,

a integração de técnicas como prompt engineering e representações estruturais dos dados tem se mostrado eficaz para potencializar o desempenho desses modelos.

Entretanto, apesar dos avanços, a literatura ainda apresenta limitações no que se refere à aplicação dessas abordagens em domínios específicos, como o de softwares educacionais. A maioria dos estudos concentra-se em contextos gerais de Engenharia de Software, sem considerar as particularidades desse domínio, como a heterogeneidade dos usuários e a natureza dos relatos de erros.

Diante disso, identifica-se uma lacuna de pesquisa relevante, relacionada à investigação da aplicabilidade dos LLMs na classificação de bugs em softwares educacionais, especialmente durante os períodos de implantação e produção. Essa lacuna justifica a realização do presente estudo, que busca contribuir para o avanço do conhecimento na área e para o desenvolvimento de soluções mais eficientes no processo de manutenção de software.

# 8 METODOLOGIA

A presente pesquisa será conduzida com o objetivo de investigar a utilização de Large Language Models (LLMs) na classificação automática de textos de relatos de erros registrados em sistemas de Help Desk de apoio à implantação e produção de softwares educacionais. Para isso, será utilizada uma abordagem metodológica alinhada aos objetivos propostos no estudo, combinando revisão integrativa da literatura, desenvolvimento experimental de protótipo e análise quanti-qualitativa dos resultados obtidos.

A metodologia adotada foi definida considerando a necessidade de compreender o estado atual das pesquisas sobre LLMs aplicados à Engenharia de Software, bem como verificar a viabilidade prática da utilização desses modelos no contexto de classificação e triagem de chamados em softwares educacionais.

## 8.1 CLASSIFICAÇÃO DA PESQUISA

Quanto à natureza, a pesquisa caracteriza-se como aplicada, pois busca investigar e utilizar recursos tecnológicos contemporâneos para auxiliar no processo de classificação de relatos de erros em sistemas de Help Desk.

Quanto aos objetivos, a pesquisa possui caráter exploratório e descritivo. É exploratória porque investiga uma temática ainda recente na Engenharia de Software, relacionada ao uso de LLMs na classificação automática de bugs e chamados. Também é descritiva porque procura observar, registrar e comparar os resultados gerados pelos modelos de inteligência artificial utilizados durante os testes realizados na pesquisa.

Quanto à abordagem dos dados, a pesquisa adota natureza quanti-qualitativa. A abordagem quantitativa será utilizada para analisar indicadores relacionados ao percentual de classificações corretas, quantidade de respostas inconclusivas, consumo de tokens e custo estimado das requisições realizadas pelos modelos de IA. Já a abordagem qualitativa será empregada para analisar a coerência das respostas, a adequação das classificações ao contexto dos chamados e as justificativas geradas pelos modelos.

Quanto aos procedimentos técnicos, a pesquisa será desenvolvida por meio de revisão integrativa da literatura, desenvolvimento experimental de protótipo e análise comparativa entre modelos de linguagem e estratégias de prompts.

## 8.2 PROCEDIMENTOS METODOLÓGICOS

A primeira etapa da pesquisa consistirá em uma Revisão Integrativa da Literatura (RIL), com o objetivo de identificar estudos relacionados ao uso de LLMs aplicados à classificação de bugs, triagem automática de chamados e manutenção de software em ambientes educacionais.

A revisão será conduzida em bases científicas reconhecidas, como IEEE Xplore, ACM Digital Library, Springer, ScienceDirect e Google Scholar.

As estratégias de busca utilizarão combinações de palavras-chave relacionadas ao tema da pesquisa, incluindo:

- `LLM AND "bug classification" AND "educational software"`
- `("software bugs" OR "bug reports") AND ("large language models") AND ("educational software")`
- `(LLM OR "large language models") AND (bug report classification) OR (bug triage)`
- `("Bug Tracking") OR (rastreamento de bugs) OR (relatórios de bugs) OR (Rastreamento de bugs impulsionado por IA)`

Os critérios de inclusão definidos para seleção dos trabalhos serão:

– artigos publicados entre 2020 e 2026;
– estudos relacionados ao uso de LLMs aplicados à Engenharia de Software;
– pesquisas relacionadas à classificação, triagem ou rastreamento de bugs;
– artigos relacionados a softwares educacionais ou sistemas de e-learning;
– estudos empíricos, revisões sistemáticas ou propostas de frameworks;
– publicações em periódicos ou conferências reconhecidas;
– trabalhos disponíveis em texto completo;
– artigos publicados em inglês ou português.

Os critérios de exclusão serão:

– artigos publicados antes de 2020;
– estudos não relacionados à Engenharia de Software ou rastreamento de bugs;

– trabalhos duplicados ou versões preliminares;
– estudos sem metodologia clara ou sem resultados validados;
– resumos, pôsteres ou apresentações sem texto completo;
– publicações não revisadas por pares, exceto trabalhos relevantes disponíveis no arXiv.

Após a seleção dos estudos, será realizada a leitura exploratória, seletiva e analítica dos trabalhos incluídos, buscando identificar abordagens utilizadas, modelos empregados, estratégias de prompt e resultados obtidos em pesquisas semelhantes.

## 8.3 POPULAÇÃO E AMOSTRA

A população da pesquisa corresponde a relatos textuais de erros registrados em sistemas de Help Desk relacionados a softwares educacionais.

A amostra será composta por chamados utilizados durante os testes da plataforma LinkNix. Esses chamados poderão ser obtidos por meio de exemplos simulados, conjuntos organizados pelos pesquisadores e registros provenientes do sistema de apoio Help Desk JEDi Educa como estudo de caso.

Cada chamado deverá conter, no mínimo, um identificador e uma descrição textual do problema relatado. Exemplo:

```
{ "id": 1, "descricao": "Usuário informa que não
consegue acessar a plataforma mesmo utilizando a senha
correta." }
```

Os critérios de inclusão da amostra serão:
– chamados escritos em linguagem natural;
– relatos relacionados a falhas, erros, dúvidas ou suporte técnico;
– textos com contexto suficiente para permitir classificação;
– chamados associados a uma categoria esperada para comparação dos resultados.

Os critérios de exclusão serão:
– textos vazios ou incompletos;
– chamados duplicados;
– registros sem contexto suficiente para classificação;
– chamados sem relação com suporte técnico ou software educacional.

## 8.4 INSTRUMENTOS DE COLETA DE DADOS

Os dados utilizados durante a pesquisa serão obtidos por meio de:

– bases bibliográficas selecionadas na revisão integrativa;

– chamados simulados e organizados pelos pesquisadores;

– registros gerados pela própria plataforma durante os testes;

– resultados retornados pelos modelos de inteligência artificial.

A plataforma LinkNix será desenvolvida utilizando as seguintes tecnologias:

– Ambiente de Desenvolvimento: Visual Studio Code;

Spring Boot ou FastAPI;

– Linguagem de programação: Java, caso seja utilizado Spring Boot; ou Python, caso seja utilizado FastAPI;

– Comunicação: API REST utilizando requisições HTTP;

dashboard: Angular;

dashboard;

– Astah: criação do diagrama de classe e caso de uso;

– Canva: logomarca e diagrama arquitetural;

– Banco de dados: PostgreSQL;

– Infraestrutura: Docker e Docker Compose;

reverse proxy: NGINX;

– Controle de versão: Git e GitHub;

– Testes e validação da API: Postman.

Os modelos de inteligência artificial utilizados na pesquisa serão:

– OpenAI GPT;

– Claude;

– DeepSeek.

## 8.5 PROCEDIMENTOS DE ANÁLISE DOS DADOS

A plataforma LinkNix será estruturada como uma API independente, permitindo integração com diferentes sistemas de Help Desk. O funcionamento da solução ocorrerá da seguinte forma:

– o sistema de Help Desk envia o chamado para a API;

– a API recebe e organiza os dados textuais;
– um mecanismo de engenharia de prompt prepara as instruções enviadas ao modelo de IA;
– o modelo processa o texto e retorna a classificação;
– os resultados são armazenados no banco de dados;
– a classificação é devolvida ao sistema solicitante.

A API registrará informações relacionadas às classificações realizadas, incluindo: categoria atribuída; nível de confiança; justificativa da classificação; modelo utilizado; quantidade de tokens consumidos; e custo estimado da requisição.

## 8.6 ENGENHARIA DE PROMPT

Durante os testes serão utilizadas diferentes estratégias de engenharia de prompt, buscando analisar como a estrutura das instruções influencia os resultados gerados pelos modelos de IA. Entre as estratégias utilizadas destacam-se:

– prompt simples (zero-shot);
– prompt com regras (rule-based);
– prompt com exemplos (few-shot);
– prompt com persona especializada.

Os prompts serão enviados aos modelos OpenAI GPT, Claude e DeepSeek utilizando o mesmo conjunto de chamados, permitindo comparação entre os resultados obtidos.

## 8.7 PROCEDIMENTOS DE ANÁLISE DOS RESULTADOS

Após o desenvolvimento da plataforma e integração com os modelos de IA, serão realizados testes utilizando os chamados definidos na amostra da pesquisa. Os resultados serão analisados por meio de procedimentos quantitativos e qualitativos.

A análise quantitativa considerará:

– quantidade total de chamados classificados;
– quantidade de classificações corretas;
– percentual de acerto;
– consumo de tokens;

– custo estimado das requisições;
– quantidade de respostas inconclusivas.

A análise qualitativa considerará:
– coerência das justificativas geradas;
– clareza das respostas;
– adequação da classificação ao contexto do chamado;
– diferenças entre modelos de IA;
– diferenças entre estratégias de prompt.

Os resultados obtidos serão organizados em tabelas e gráficos comparativos, permitindo observar diferenças entre os modelos utilizados e as estratégias de prompt empregadas durante os testes.

## 8.8 JUSTIFICATIVA DA METODOLOGIA

A modelagem arquitetural da solução, os diagramas UML e os requisitos funcionais e não funcionais foram incorporados à metodologia da pesquisa por representarem elementos diretamente relacionados ao desenvolvimento experimental da plataforma LinkNix. Dessa forma, a metodologia contempla não apenas os procedimentos científicos utilizados na revisão integrativa da literatura, mas também as etapas de modelagem, arquitetura, implementação e validação da solução proposta.

A metodologia escolhida mostra-se adequada aos objetivos da pesquisa, pois combina fundamentação teórica, desenvolvimento experimental e análise comparativa de resultados relacionados ao uso de Large Language Models na classificação de relatos de erros em softwares educacionais. A revisão integrativa da literatura possibilita compreender o estado atual das pesquisas sobre LLMs aplicados à Engenharia de Software. O desenvolvimento da plataforma LinkNix permite aplicar os conceitos estudados em um contexto prático de classificação automática de chamados. Já a análise quanti-qualitativa possibilita observar tanto aspectos objetivos quanto subjetivos do desempenho dos modelos.

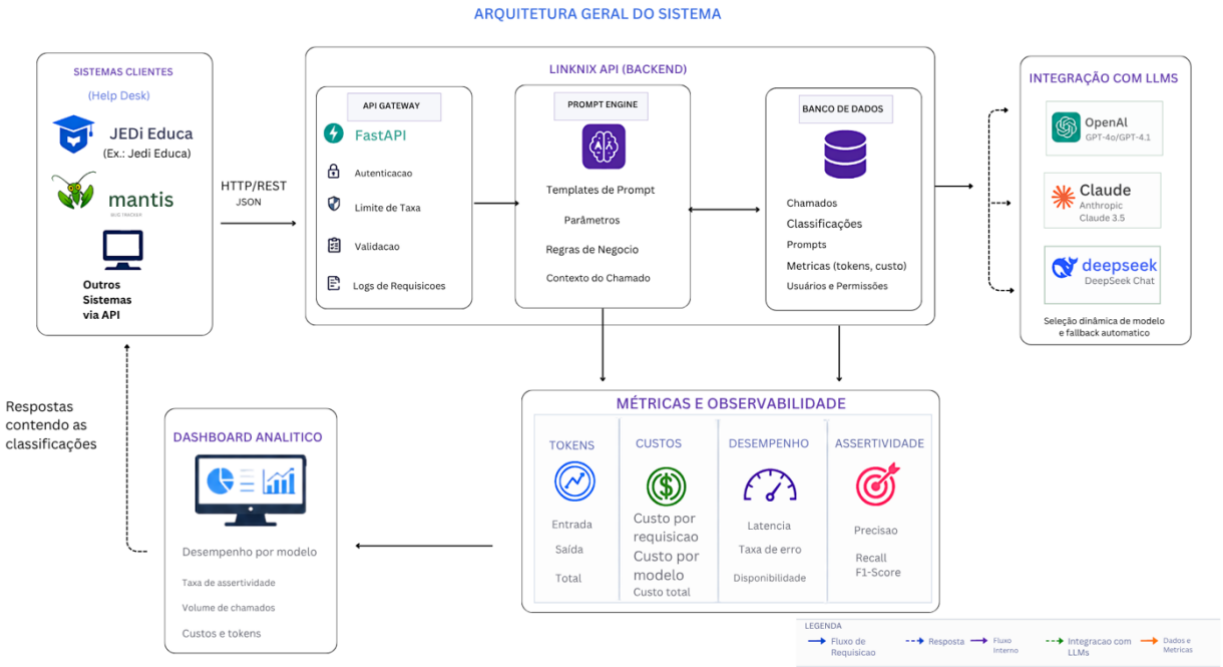
## 8.9 ARQUITETURA DA SOLUÇÃO

A plataforma LinkNix foi desenvolvida com base em uma arquitetura modular e escalável, projetada para suportar a integração de múltiplos modelos de inteligência artificial generativa no processo de classificação automática de chamados técnicos.

### 8.9.1 Arquitetura geral da plataforma

A arquitetura geral da plataforma LinkNix é composta por módulos independentes que se comunicam por meio de APIs REST. A Figura 1 apresenta a visão geral da arquitetura proposta.

Figura 1 – Arquitetura geral da plataforma LinkNix




Fonte: Elaborada pelos autores.(2026)

### 8.9.2 Sistema clientes LinkNix

O módulo de Sistema Clientes LinkNix representa a camada de abstração responsável por gerenciar os clientes que utilizam a plataforma. Essa abordagem desacoplada permite reutilização da plataforma em diferentes cenários organizacionais.

### 8.9.3 Sistemas clientes (Help Desk)

Os sistemas clientes correspondem às plataformas de suporte técnico que integram a API LinkNix para envio e recebimento de classificações automáticas de chamados. Essa abordagem desacoplada permite reutilização da plataforma em diferentes cenários organizacionais.

### 8.9.4 API Gateway

A camada de API Gateway representa o ponto central de entrada das requisições realizadas pelos sistemas clientes. Esse componente é responsável pelo roteamento, autenticação e validação das requisições antes de encaminhá-las ao módulo de processamento.

### 8.9.5 Prompt Engine

O Prompt Engine constitui o núcleo inteligente da plataforma LinkNix. Esse módulo é responsável pela construção dinâmica dos prompts enviados aos modelos de linguagem. Seu funcionamento baseia-se em engenharia de prompt parametrizável, permitindo modificar instruções sem necessidade de alterar diretamente o código-fonte da aplicação.

Entre as funcionalidades previstas para esse módulo destacam-se:

– utilização de templates de prompts;
– definição de regras de negócio;
– parametrização contextual;
– construção dinâmica de instruções;
– inclusão de exemplos de classificação;
– utilização de personas especializadas;
– adaptação do prompt conforme o modelo de IA selecionado.

O Prompt Engine também será responsável por incorporar contexto adicional relacionado ao chamado recebido, aumentando a precisão semântica das respostas produzidas pelos modelos de linguagem.

### 8.9.6 Integração com grandes modelos de linguagem (LLMs)

A plataforma LinkNix foi projetada para suportar múltiplos provedores de inteligência artificial generativa, permitindo comparação experimental entre diferentes

modelos de linguagem. Os modelos inicialmente previstos na pesquisa são: OpenAI GPT-4o/GPT-4.1; Claude 3.5; e DeepSeek Chat.

A integração ocorrerá por meio de APIs oficiais disponibilizadas pelos provedores, utilizando requisições autenticadas via tokens de acesso. A arquitetura permite: seleção dinâmica de modelos; fallback automático em caso de indisponibilidade; comparação entre respostas; monitoramento de custos; análise de consumo de tokens; e avaliação de desempenho e assertividade.

### 8.9.7 Banco de dados

O módulo de persistência será implementado utilizando PostgreSQL, escolhido devido à sua robustez, confiabilidade e suporte avançado a operações relacionais. O banco de dados será responsável pelo armazenamento de: chamados recebidos; classificações realizadas; prompts utilizados; usuários e permissões; métricas operacionais; consumo de tokens; custos estimados; e histórico de execuções.

### 8.9.8 Módulo de métricas e observabilidade

A arquitetura da LinkNix incorpora um módulo específico de métricas e observabilidade, responsável pelo monitoramento operacional da plataforma. As métricas previstas incluem:

**a) Métricas de consumo:** tokens de entrada, tokens de saída e total de tokens consumidos;

**b) Métricas financeiras:** custo por requisição, custo por modelo e custo acumulado;

**c) Métricas de desempenho:** disponibilidade e taxa de erro;

**d) Métricas de assertividade:** precisão, recall, F1-score e percentual de classificações corretas.

Essas informações permitem avaliar não apenas a qualidade das respostas geradas pelos modelos, mas também a viabilidade operacional e financeira da utilização de LLMs em processos de manutenção de software.

### 8.9.9 Dashboard analítico

O Dashboard Analítico foi concebido para fornecer visualização centralizada das informações produzidas pela plataforma. O painel permitirá acompanhamento de: desempenho dos modelos; volume de chamados; métricas de assertividade; custos

operacionais; consumo de tokens; e indicadores de utilização. A interface poderá ser desenvolvida utilizando Angular, permitindo construção de componentes gráficos interativos e integração em tempo real com a API.

### 8.9.10 Fluxo operacional da classificação

O fluxo de funcionamento da plataforma ocorre da seguinte forma:

– o sistema cliente envia um chamado técnico para a API LinkNix;
– o API Gateway valida e processa os dados recebidos;
– o Prompt Engine monta dinamicamente o prompt;
– o modelo de IA selecionado recebe o prompt;
– o modelo retorna a classificação do chamado;
– a API registra métricas e resultados no banco de dados;
– o resultado é devolvido ao sistema cliente;
– as informações ficam disponíveis no dashboard analítico.

Esse fluxo permite automatizar parcialmente o processo de triagem de chamados técnicos, reduzindo esforço manual e aumentando a velocidade de classificação.

### 8.9.11 Considerações arquiteturais

A arquitetura da LinkNix foi concebida seguindo princípios modernos de Engenharia de Software, priorizando: modularidade; escalabilidade; desacoplamento; reutilização; interoperabilidade; observabilidade; e facilidade de manutenção.

Além disso, a utilização de uma arquitetura orientada a APIs permite evolução futura da plataforma, possibilitando integração com novos modelos de inteligência artificial, inclusão de mecanismos de aprendizado contínuo e expansão para outros domínios além do contexto educacional.

## 8.10 MODELAGEM UML

A modelagem UML (Unified Modeling Language) foi utilizada neste trabalho com o objetivo de representar graficamente a estrutura, os comportamentos e os fluxos operacionais da plataforma LinkNix. Os diagramas desenvolvidos contemplam diferentes perspectivas do sistema, incluindo visão funcional, estrutural e comportamental da aplicação.

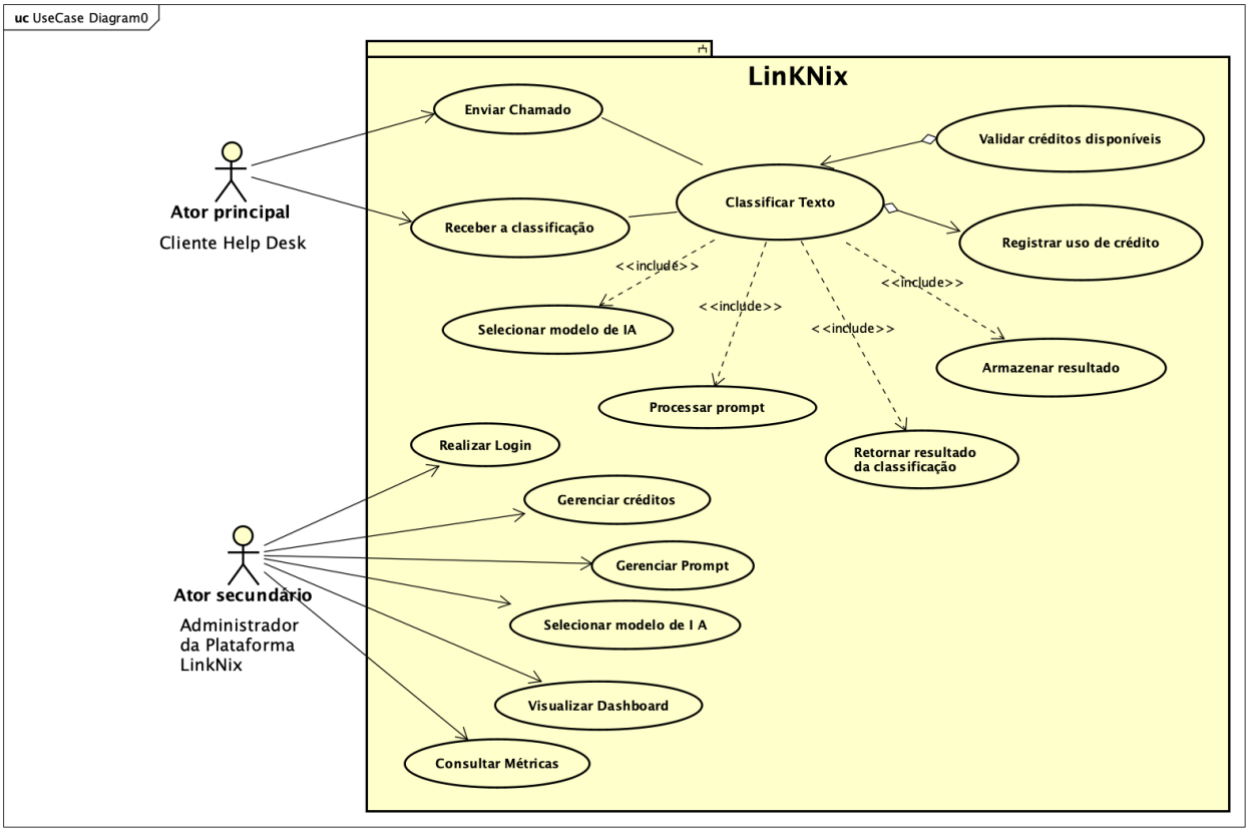
### 8.10.1 Diagrama de casos de uso

O diagrama de casos de uso representa as principais funcionalidades disponibilizadas pela plataforma LinkNix, bem como os atores envolvidos na interação com o sistema. O modelo apresenta dois atores principais:

**a) Ator principal – Cliente Help Desk:** representa os sistemas externos responsáveis pelo envio dos chamados técnicos para classificação automática. As principais interações realizadas por esse ator incluem: enviar chamados técnicos; receber classificações automáticas; solicitar processamento textual; e receber o resultado da classificação.

**b) Ator secundário – Administrador da plataforma:** representa o usuário responsável pelo gerenciamento operacional da plataforma LinkNix. Entre as funcionalidades associadas a esse ator destacam-se: realização de autenticação no sistema; gerenciamento de créditos; gerenciamento de prompts; seleção de modelos de IA; visualização do dashboard; e consulta de métricas operacionais.

Figura 2 – Diagrama de Casos de Uso da Plataforma LinkNix



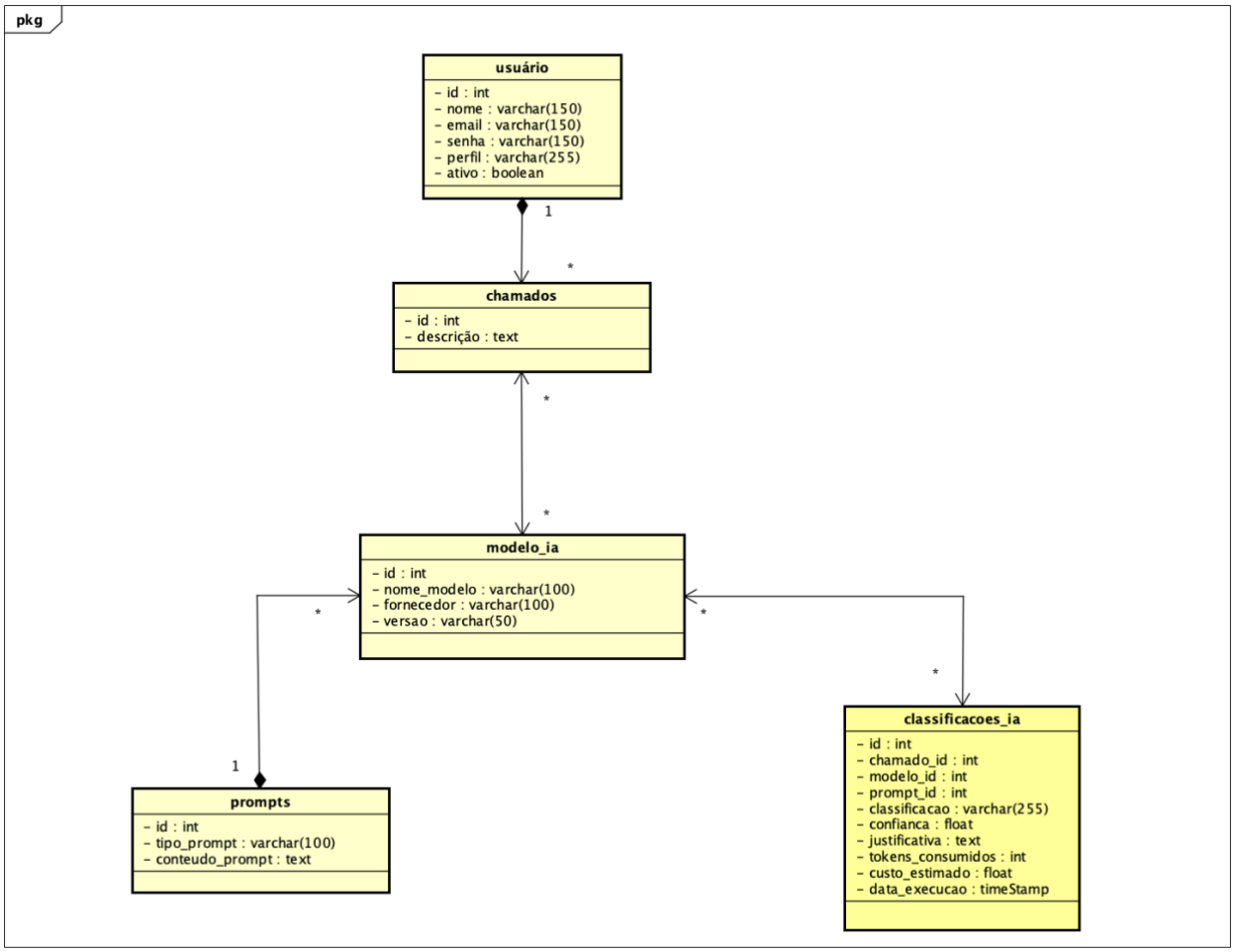

Fonte: Elaborada pelos autores.(2026)

### 8.10.2 Diagrama de classes

O diagrama de classes da plataforma LinkNix representa a estrutura lógica das principais entidades responsáveis pelo funcionamento da solução proposta. A modelagem foi elaborada com o objetivo de organizar os dados relacionados ao processo de classificação automática de chamados utilizando LLMs.

A estrutura modelada contempla entidades relacionadas ao gerenciamento de usuários, chamados técnicos, modelos de inteligência artificial, prompts utilizados durante o processamento e armazenamento dos resultados das classificações realizadas. A entidade classificacoes_ia constitui o núcleo operacional da modelagem, sendo responsável pelo registro dos resultados gerados pelos modelos de inteligência artificial, incluindo: classificação atribuída; nível de confiança; justificativa da IA; quantidade de tokens consumidos; custo estimado; e data de execução.

Figura 3 – Diagrama de Classes da Plataforma LinkNix




Fonte: Elaborada pelos autores.(2026)

A modelagem foi estruturada seguindo princípios de normalização e organização relacional, contribuindo para escalabilidade, manutenção e integridade dos dados armazenados pela API LinkNix.

## 8.11 REQUISITOS FUNCIONAIS E NÃO FUNCIONAIS

Quadro 1 – Requisitos Funcionais da Plataforma LinkNix

| Código | Requisito | Descrição |
|---|---|---|
| RF01 | Cadastro de Usuários | O sistema deve permitir cadastro de usuários. |
| RF02 | Autenticação de Usuários | O sistema deverá permitir login utilizando autenticação JWT. |
| RF03 | Controle de acesso | O sistema deverá possuir controle de acesso baseado em perfis. |
| RF04 | Cadastro de chamados | O sistema deverá permitir abertura de chamados contendo id e descrição. |
| RF05 | Integração com modelos de IA | O sistema deverá enviar chamados para modelos de IA (OpenAI GPT, Claude, DeepSeek). |
| RF06 | Comparação entre Modelos LLM | O sistema deverá comparar métricas e resultados produzidos por diferentes modelos LLM. |
| RF07 | Engenharia de Prompt | O sistema deverá permitir utilização de diferentes estratégias de prompt (Zero-shot, Rule-based, Few-shot e Persona-based Prompting). |
| RF08 | Registro das classificações | O sistema deverá armazenar categoria atribuída, nível de confiança, modelo utilizado, prompt utilizado, tokens consumidos, custo estimado e justificativa da IA. |
| RF09 | Dashboard de métricas | O sistema deverá exibir métricas comparativas: percentual de acertos, consumo de tokens, custos, tempo de resposta e qualidade de classificações. |
| RF10 | API Rest | O sistema deverá disponibilizar endpoints REST para integração externa. |
| RF11 | Consulta saldo/extrato | O sistema deve permitir gerenciamento de créditos. |

Fonte: Elaborado pelos autores (2026).

Quadro 2 – Requisitos Não Funcionais da Plataforma LinkNix

| Código | Requisito | Descrição |
|---|---|---|
| RNF01 | Segurança | O sistema deverá utilizar JWT, BCrypt, controle de permissões e HTTPS. |
| RNF02 | Infraestrutura | A aplicação deverá ser executada em containers Docker. |
| RNF03 | Portabilidade | O sistema deve permitir integração com softwares externos. |
| RNF04 | Desempenho | A API deverá responder às requisições em tempo adequado para processamento e classificação dos chamados. |

| Código | Requisito | Descrição |
|---|---|---|
| RNF05 | Rastreabilidade | O sistema deverá registrar logs de todas as classificações realizadas. |
| RNF06 | Manutenibilidade | O código deverá seguir arquitetura em camadas e princípios de Clean Code. |
| RNF07 | Banco de Dados | O sistema deverá utilizar PostgreSQL ou MySQL para persistência das informações. |
| RNF08 | Arquitetura de Software | O sistema deverá possuir API REST para comunicação entre frontend, backend e serviços externos. |
| RNF09 | Disponibilidade | O sistema deverá manter disponibilidade adequada durante o processamento das requisições. |

Fonte: Elaborado pelos autores (2026).

## 9 CRONOGRAMA

O cronograma da pesquisa foi elaborado considerando as etapas necessárias para desenvolvimento teórico, arquitetural, experimental e documental da plataforma LinkNix. As atividades foram distribuídas ao longo do período de execução do Trabalho de Conclusão de Curso, contemplando revisão bibliográfica, modelagem da solução, implementação da API, realização de testes experimentais e redação científica do trabalho.

Quadro 3 – Cronograma de execução da pesquisa

| ETAPAS | Fev | Mar | Abr | Mai | Jun | Jul | Ago | Set | Out | Nov | Dez |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Levantamento bibliográfico | • | • | | | | | | | | | |
| Revisão integrativa da literatura | | • | • | • | | | | | | | |
| Definição da arquitetura da solução | | • | • | • | | | | | | | |
| Modelagem UML | | | | • | | | | | | | |
| Requisitos funcionais e não funcionais | | | | • | | | | | | | |
| Desenvolvimento da API LinkNix | | | | | • | • | • | • | • | | |
| Integração com modelos de IA | | | | | | | • | • | | | |
| Desenvolvimento do dashboard | | | | | | • | | | | | |
| Implementação da engenharia de prompt | | | | | • | • | | | | | |
| Testes experimentais | | | | | | | • | • | | | |
| Coleta e análise de métricas | | | | | | | | • | | | |
| Escrita dos resultados | | | | | | | | • | • | | |
| Revisão final e adequação ABNT | | | | | | | | | • | | |
| Entrega do TCC | | | | | | | | | • | | |

Fonte: Elaborado pelos autores (2026).

## 10 CUSTOS

A presente pesquisa envolve predominantemente recursos computacionais e ferramentas de desenvolvimento de software. Dessa forma, os custos previstos concentram-se principalmente na utilização de serviços de inteligência artificial, infraestrutura computacional e ferramentas auxiliares utilizadas durante o desenvolvimento experimental da plataforma LinkNix.

Grande parte das tecnologias utilizadas possui licenciamento gratuito ou acadêmico, reduzindo significativamente os custos operacionais da pesquisa. Entretanto, alguns serviços relacionados à utilização de LLMs podem gerar custos variáveis conforme o volume de requisições realizadas durante os testes experimentais.

Quadro 4 – Estimativa de custos da pesquisa

| Item | Descrição | Valor Estimado |
|---|---|---|
| OpenAI API | Utilização do modelo GPT para testes experimentais | R$ 150,00 |
| Claude API | Utilização do modelo Claude para classificações | R$ 100,00 |
| DeepSeek API | Utilização experimental do modelo DeepSeek | R$ 50,00 |
| Infraestrutura Docker | Ambiente local de containers | R$ 0,00 |
| PostgreSQL | Banco de dados open source | R$ 0,00 |
| FastAPI | Framework backend open source | R$ 0,00 |
| Angular | Desenvolvimento do dashboard | R$ 0,00 |
| GitHub | Controle de versão | R$ 0,00 |
| Figma | Prototipação das interfaces | R$ 0,00 |
| Astah UML | Modelagem UML acadêmica | R$ 0,00 |
| Canva | Criação de diagramas e elementos visuais | R$ 0,00 |
| Equipamentos computacionais | Notebook pessoal utilizado no desenvolvimento | Recurso próprio |

Fonte: Elaborado pelos autores (2026).

Ressalta-se que os custos apresentados possuem caráter estimativo, podendo variar conforme a quantidade de testes realizados, volume de chamadas às APIs de inteligência artificial e alterações nos modelos de precificação dos provedores utilizados durante a execução da pesquisa.

## 11 RESULTADOS ESPERADOS

Espera-se que a presente pesquisa alcance resultados compatíveis com os objetivos propostos, demonstrando a viabilidade do uso de grandes modelos de linguagem na classificação automática de relatos de erros registrados em sistemas de Help Desk voltados a softwares educacionais.

Em relação aos objetivos específicos iniciais, espera-se consolidar referencial teórico sobre os conceitos de bug tracking, bug triaging, manutenção de software e aplicação de LLMs em processos de suporte técnico. Também se pretende identificar, por meio da revisão integrativa da literatura, o estado atual das pesquisas relacionadas ao tema, bem como lacunas científicas ainda existentes.

No que se refere aos objetivos práticos, espera-se propor, desenvolver e validar a API protótipo LinkNix, destinada à classificação automatizada de chamados. Pretende-se que a solução seja integrada ao sistema de Help Desk JEDi Educa em softwares educacionais, permitindo apoio ao processo inicial de triagem e encaminhamento das demandas registradas pelos usuários.

Como contribuição para o avanço do conhecimento, espera-se ampliar as discussões acadêmicas sobre o uso de Inteligência Artificial na manutenção de software, especialmente no contexto de softwares educacionais, área ainda em desenvolvimento e com potencial de crescimento científico e tecnológico.

Entre os benefícios práticos esperados, destacam-se a redução do esforço manual na classificação de chamados, maior agilidade no atendimento inicial, padronização do processo de triagem e possibilidade de evolução futura da ferramenta para outros contextos organizacionais.

Por fim, espera-se que os resultados obtidos permitam verificar a hipótese de que modelos de linguagem podem ser aplicados de forma funcional e integrável a sistemas reais de Help Desk, servindo como apoio às equipes responsáveis pelo suporte e manutenção de software.

# REFERÊNCIAS

GIRAY, Görkem et al. On the use of deep learning in software defect prediction. *Journal of Systems and Software*, v. 195, p. 111537, jan. 2023. Disponível em: https://doi.org/10.1016/j.jss.2022.111537. Acesso em: 30 mar. 2026.

HOU, Xinyi et al. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Transactions on Software Engineering and Methodology*, v. 33, n. 8, p. 1–79, 30 nov. 2024. Disponível em: https://doi.org/10.1145/3695988. Acesso em: 30 mar. 2026.

KOYUNCU, Anil. Exploring Fine-Grained Bug Report Categorization with Large Language Models and Prompt Engineering: An Empirical Study. *ACM Transactions on Software Engineering and Methodology*, v. 35, n. 3, p. 1–47, 31 mar. 2026.

MOURA, Adelina; CARVALHO, Ana Amélia A. Literacia de Prompts para Potenciar o Uso da Inteligência Artificial na Educação. *RE@D - Revista de Educação a Distância e eLearning*, v. 6, n. 2, 2023.

PRESSMAN, Roger S.; MAXIM, Bruce R. *Engenharia de Software: uma abordagem profissional*. 8. ed. Porto Alegre: AMGH, 2016.

PUSHPARANI, M. K. et al. The Evolution of Bug Tracking Systems: A Review of Advancements in Software Quality. *International Research Journal on Advanced Engineering and Management (IRJAEM)*, v. 3, n. 09, p. 2915–2918, 22 set. 2025. Disponível em: https://doi.org/10.47392/IRJAEM.2025.0460. Acesso em: 30 mar. 2026.

SANTOS, Igor Ramsés Temóteo et al. Um Mapeamento Sistemático sobre Técnicas de Classificação e Categorização de Bugs em Software. *Revista Eletrônica de Iniciação Científica em Computação*, v. 23, p. 298–306, 31 dez. 2025.

SHAO, Shuai; XIAO, Lu; YU, Tingting. Identifying Concurrency Bug Reports via Linguistic Patterns. *arXiv*, 22 jan. 2026. Disponível em: http://arxiv.org/abs/2601.16338. Acesso em: 30 mar. 2026.

SIMÃO, José Pedro Schardosim et al. Inteligência Artificial Agêntica na Educação: Um Agente Conversacional para Construção de Competências na Educação Básica. In: SIMPÓSIO BRASILEIRO DE INFORMÁTICA NA EDUCAÇÃO, 36., 2025. *Anais do XXXVI Simpósio Brasileiro de Informática na Educação (SBIE 2025)*. Brasil: Sociedade Brasileira de Computação – SBC, 24 nov. 2025. Disponível em: https://sol.sbc.org.br/index.php/sbie/article/view/38434. Acesso em: 30 mar. 2026.

SOMMERVILLE, Ian. *Engenharia de Software*. 10. ed. São Paulo: Pearson, 2019.

YILMAZ, Halit Eyüp; TOROSLU, İsmail Hakkı; KÖKSAL, Ömer. A Comparative Study of Contemporary Learning Paradigms in Bug Report Priority Detection. *IEEE Access*, v. 12, p. 126577–126586, 2024. Disponível em: https://doi.org/10.1109/ACCESS.2024.3451125. Acesso em: 30 mar. 2026.